

ESIC HMS — Deployment Guide & Infrastructure Audit

ESIC HMS — Deployment Guide & Infrastructure Audit

1. What this system actually is

2. System requirements

3. Environment variables — ground truth

4. Build & run — per app

5. Infrastructure decisions that need a deliberate answer (not just "deploy it")

6. Security audit findings — fix before go-live

7. Observability & operations

8. Database operations

9. Suggested deployment topology (based on the above)

10. Go-live checklist

ESIC HMS — Deployment Guide & Infrastructure Audit

Derived directly from the current codebase (package manifests, Dockerfiles, `docker-compose.yml`, `prisma/schema.prisma`, `main.ts`, guards, middleware, and every `process.env.*` read in `apps/api/src`) as of **2026-09-17**. Where the code disagrees with what's declared in config files (e.g. an env var that's documented but never read), that gap is called out explicitly so it doesn't get silently deployed on faith.

1. What this system actually is

A pnpm/Turborepo monorepo with two deployable apps:

App	Path	Framework	Purpose
api	apps/api	NestJS 10 (Express adapter) + Prisma 5	REST API, <code>/api</code> prefix, 21 domain modules (employee, patient, OPD, IPD/admission, pharmacy, inventory, procurement, billing, lab, therapy, analytics, reports, RBAC admin, etc.)
web	apps/web	React 18 + Vite 5 + Tailwind	SPA frontend, calls the API via <code>VITE_API_URL</code>

Data layer: **PostgreSQL 15** via Prisma (59 models, 11 migrations at last count). This is a real system of record for patient/employee health data — treat it as sensitive PII/PHI, not a toy app.

2. System requirements

2.1 Build/runtime toolchain (must match exactly — pinned in repo)

Requirement	Version	Source
Node.js	20.x	<code>.nvmrc</code> , <code>package.json engines.node: ">=20.0.0"</code> , both Dockerfiles (<code>node:20-*</code>)
pnpm	9.15.4 (via Corepack)	<code>package.json packageManager</code> , both Dockerfiles, CI
PostgreSQL	15	<code>docker-compose.yml</code> , <code>prisma/schema.prisma</code> (<code>provider = "postgresql"</code>)
Redis	7 (provisioned, see §5.1 — currently unused)	<code>docker-compose.yml</code> , <code>.env.example</code>

2.2 Minimum server sizing (production, single-region)

There's no existing production infra to benchmark against, so these are reasoned from what the code actually does — adjust after real load testing:

Component	CPU	RAM	Disk	Why
API container	1–2 vCPU	1.5–2 GB min	1 GB (image + logs)	Puppeteer launches a real headless Chromium and keeps one shared browser instance alive for the life of the process (<code>DocumentRenderService</code> , §5.3) — budget 300–500 MB for Chromium alone, on top of Node/Nest. Undersizing this is the single most likely cause of OOM kills in production.
PostgreSQL	1–2 vCPU	2–4 GB	Start at 20 GB, monitor growth	Every mutating request (<code>POST/PUT/PATCH/DELETE</code>) writes a row via <code>AuditInterceptor</code> — audit log volume grows continuously and has no visible retention/archival job in the code. This table will be your fastest-growing one; plan a retention policy before it's a production fire drill.
Web (static)	—	—	~5–20 MB build output	Pure static assets after <code>vite build</code> ; any static host/CDN works.
Redis (if you decide to keep it — see §5.1)	0.5 vCPU	256–512 MB	negligible	Nothing currently uses it, so this is speculative sizing only.

For a low-traffic single-facility rollout, a single 2 vCPU / 4 GB VM running API + Postgres via Docker Compose is plausible to start; scale out once you have real usage numbers.

2.3 Local/dev machine requirements (for anyone building this)

- Docker + Docker Compose (the only documented local workflow — `docker-compose.yml`)
- Node 20, pnpm 9.15.4 (Corepack: `corepack enable`)
- ~2 GB free disk for `node_modules` + Puppeteer's bundled Chromium download (happens on `pnpm install` for `apps/api` , since `puppeteer` — not `puppeteer-core` — is a dependency)

3. Environment variables — ground truth

Two sources exist (`/.env.example` and `apps/api/.env.example`) but **what the API actually reads** is narrower — confirmed by grepping every `process.env.*` reference in `apps/api/src` :

Variable	Actually read by code?	Required in prod	Default if unset	Notes
DATABASE_URL	✅ (via Prisma)	Yes	none — hard fail	postgresql://user:pass@host:5432/db
API_PORT	✅ main.ts	No	3000	Only used in the non-Vercel bootstrapLocal() path
JWT_ACCESS_SECRET	✅ auth.service.ts, jwt.strategy.ts	Yes — critical	'dev_jwt_access_secret_key_12345'	⚠️ See §6.1 — silently falls back to a hardcoded, publicly-visible-in-this-repo secret if unset.
JWT_REFRESH_SECRET	✅ auth.service.ts	Yes — critical	'dev_jwt_refresh_secret_key_67890'	Same failure mode as above.
JWT_EXPIRES_IN	✅ auth.service.ts	No	'8h'	Not present in either .env.example file — undocumented. Refresh token expiry is separately hardcoded to 7d and is not configurable.
VERCEL	✅ main.ts	Auto-set by platform	—	Only relevant if you deploy the API to Vercel (see §5.2 for why that's currently a bad idea).
REDIS_URL	❌ Never read anywhere in apps/api/src	No (currently)	—	Declared in both .env.example files and docker-compose.yml, wired into turbo.json globalEnv, and ioredis is a listed dependency — but nothing in the codebase constructs a Redis client. See §5.1.
WEB_PORT	❌ Not read by API code (Vite's own server.port is hardcoded to 5173 in vite.config.ts, not env-driven)	No	—	Documented but effectively dead.
VITE_API_URL	✅ Web app (apps/web/src/api/client.ts, useAuth.ts)	Yes, at build time	http://localhost:3000	⚠️ Vite inlines this at build time, not runtime — see §6.4.
VITE_API_PROXY_TARGET	✅ vite.config.ts dev proxy only	No (dev-only)	http://127.0.0.1:3000	Never used in a production build — only Vite's dev server.
NODE_ENV	Read implicitly by NestJS/Express/Prisma internals	Recommended	—	Set to production in prod.
LABOUR_DEPT_API_BASE_URL / _API_KEY	❌ Commented out, "Phase 2+"	No	—	Not implemented yet — ignore for this deployment.
OBJECT_STORAGE_ENDPOINT / _BUCKET	❌ Commented out, "Phase 2+"	No	—	No file/object storage code exists anywhere (no Multer, S3, or MinIO usage found). Not needed yet.
SENTRY_DSN	❌ Commented out, "Phase 16"	No	—	Not implemented. No error-tracking SDK is wired in currently — see §7.3.

Action item: the two .env.example files and turbo.json list several variables the code doesn't consume (REDIS_URL, WEB_PORT) and are missing one it does (JWT_EXPIRES_IN). Reconcile these before onboarding a new deployer off the example files alone.

4. Build & run — per app

4.1 API (apps/api)

```
pnpm install --frozen-lockfile # postinstall also runs `prisma generate`
pnpm --filter @esic-hms/api prisma:generate
pnpm --filter @esic-hms/api build # runs: prisma generate && nest build → apps/api/dist
pnpm --filter @esic-hms/api start:prod # node dist/main
```

- apps/api/src/main.ts supports **two run modes from the same build**:
 - Persistent server** (bootstrapLocal()): runs whenever process.env.VERCEL is unset — this is the path a Docker/VM/container deployment uses (node dist/main, listens on API_PORT).
 - Vercel serverless handler** (index.js → dist/main.js default export): only fires when deployed on Vercel. **Read §5.2 before choosing this path** — it's currently a poor fit for this app.
- Database migrations are **not** part of the build script. Run explicitly before starting a new version:

```
npx prisma migrate deploy # production — never `migrate dev` against prod data
```

4.2 Web (apps/web)

```
pnpm install --frozen-lockfile
pnpm --filter @esic-hms/web build # tsc -b && vite build → apps/web/dist
```

Output is static files in `apps/web/dist` — deployable to any static host (Nginx, Vercel static, Cloudflare Pages, S3+CDN, etc.).

⚠ **Gap:** `apps/web/Dockerfile` **only runs** `pnpm dev` (the Vite dev server) — **there is no production Dockerfile that builds and serves the static bundle**. As written, that Dockerfile is dev-only and should not be used for a production image. You'll need to add a build stage + static server (Nginx/ `serve`) before containerizing the web app for prod. A minimal example:

```
FROM node:20-alpine AS build
WORKDIR /app
RUN corepack enable && corepack prepare pnpm@9.15.4 --activate
COPY . .
RUN pnpm install --frozen-lockfile && pnpm --filter @esic-hms/web build

FROM nginx:alpine
COPY --from=build /app/apps/web/dist /usr/share/nginx/html
# SPA fallback: rewrite unknown routes to index.html for react-router-dom
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

5. Infrastructure decisions that need a deliberate answer (not just "deploy it")

5.1 Redis is provisioned but not used — decide now, not after go-live

`docker-compose.yml`'s own comments say Redis is meant for "OPD queue token counters (atomic INCR), BullMQ job queues, and short-lived session/cache data." `ioredis` is a dependency. But a full source grep of `apps/api/src` finds **zero references to Redis, `ioredis`, or `REDIS_URL`**. Either:

- (a) it's aspirational infra for a not-yet-built feature — in which case don't pay for/operate a Redis instance in production yet, or
- (b) something (e.g. OPD token concurrency) is currently relying on a non-atomic fallback (likely a DB counter) that Redis was meant to replace — worth checking before you scale OPD concurrent registrations.

Don't provision Redis in prod "just in case" — it's unmonitored, unused infrastructure that adds attack surface and cost for nothing today.

5.2 Don't deploy the API to Vercel serverless as-is — three concrete blockers in the code

`apps/api/vercel.json` and the dual-mode `main.ts` suggest Vercel was considered. Three things in the current code make that a bad fit for a serverless function model:

1. **Persistent headless Chromium.** `DocumentRenderService` launches one Puppeteer/Chromium instance and **deliberately keeps it alive across requests** (comment in the code: avoids the ~1–2s per-launch cost). Serverless functions are ephemeral — you'd either eat a Chromium cold-start on every invocation, or (worse) leak orphaned Chromium processes across function instances. The bundled `puppeteer` package (not `puppeteer-core`) also ships a full Chromium binary, which routinely blows past serverless function size limits (Vercel Node functions: 50 MB compressed) unless swapped for `puppeteer-core` + `@sparticuz/chromium`.
2. **In-process cron.** `ScheduleModule.forRoot()` + `@Cron(CronExpression.EVERY_DAY_AT_MIDNIGHT)` in `ipd-finance.service.ts` only fires if the Node process is actually running at midnight. Serverless functions don't idle-run — this job silently never executes on Vercel unless you replace it with Vercel Cron Jobs hitting a dedicated endpoint.
3. **In-memory browser promise cache is per-instance.** Under real concurrency, serverless platforms spin up multiple parallel instances, each with its own Chromium — defeating the "one shared browser" design intent entirely and multiplying memory use.

Recommendation: run the API as a long-lived process — a container on a VM, ECS/Fargate, Render, Railway, Fly.io, etc. — not a serverless function, until the PDF rendering and cron job are re-architected for serverless (headless-browser-as-a-service, or move the cron to an external scheduler). The web app (pure static output) has no such constraint and is a fine Vercel/CDN candidate as-is.

5.3 Puppeteer/Chromium system dependencies — the current Dockerfile is missing them

`apps/api/Dockerfile` is `node:20-bookworm-slim`, which does **not** include the shared libraries Chromium needs to launch headless (`libnss3`, `libatk-bridge2.0-0`, `libgtk-3-0`, `libgbm1`, fonts, etc.). Puppeteer will fail to launch on this image as-is when a PDF (receipt/lab report/admin report) is requested. Add before `pnpm install`:

```
RUN apt-get update && apt-get install -y --no-install-recommends \
  ca-certificates fonts-liberation libasound2 libatk-bridge2.0-0 libatk1.0-0 \
  libcairo2 libcups2 libdbus-1-3 libexpat1 libgbm1 libglib2.0-0 libgtk-3-0 \
  libnspr4 libnss3 libpango-1.0-0 libx11-6 libx11-xcb1 libxcomposite1 \
  libxdamage1 libxext6 libxfixes3 libxrandr2 libxss1 libxtst6 wget \
  && rm -rf /var/lib/apt/lists/*
```

(or point `PUPPETEER_EXECUTABLE_PATH` at a system Chromium and skip Puppeteer's own download, which also shrinks the image significantly).

Also note: `apps/api/Dockerfile`'s `CMD` is `pnpm start:dev` (the watch-mode dev server) — that's a dev image, not a production one. Add a real production Dockerfile stage using `pnpm build` + `pnpm start:prod` (`node dist/main`) before shipping this container to prod.

6. Security audit findings — fix before go-live

These are concrete, code-confirmed issues, ranked by severity:

6.1 ● Critical — hardcoded fallback JWT secrets

`jwt.strategy.ts` and `auth.service.ts` both do:

```
secretOrKey: process.env.JWT_ACCESS_SECRET || 'dev_jwt_access_secret_key_12345'
```

and the same pattern for `JWT_REFRESH_SECRET`. If these env vars aren't set in production, the app **starts successfully** and silently signs/verifies tokens with a secret that's sitting in plaintext in this repository. There's no startup validation that fails loudly if they're missing. **Before any production deploy:** set both to long random values, and ideally add a startup check that refuses to boot without them.

6.2 ● Critical — seeded demo accounts use hardcoded, guessable passwords

`apps/api/prisma/seed.ts` creates real, functional accounts including a `SuperAdmin` (`superadmin@esic.gov.in / SuperAdminSecret123!`) and several role accounts (`AdminPass123!`, `DoctorPass123!`, `NursePass123!`, etc.) with `upsert` — meaning re-running the seed against prod won't even fail if they already exist. If this seed script is ever run against a production database (directly, or by mistake in a CI/CD step), these become live, publicly-guessable admin credentials. **Do not run `prisma db seed` against production.** If you need an initial admin account in prod, create a separate prod-only seed that generates a random password and forces rotation on first login, and keep the demo-account seed strictly for dev/staging.

6.3 ● High — CORS is wide open

`main.ts` calls `nestApp.enableCors()` / `app.enableCors()` with no options in both the Vercel and local bootstrap paths — this allows any origin. Lock this down to the known frontend origin(s) via `enableCors({ origin: [...], credentials: true })` before production, driven by an env var (e.g. `CORS_ORIGIN`) that doesn't currently exist in the codebase.

6.4 ● High — frontend API URL is baked in at build time, not runtime

`VITE_API_URL` is inlined into the JS bundle by Vite at build time (`import.meta.env.VITE_API_URL`). This means **you cannot point the same built artifact at different environments** (staging vs. prod) — you must produce a separate build per environment with the correct `VITE_API_URL` set at build time, or refactor to fetch config at runtime. Plan your CI/CD around "build once per environment," not "build once, promote everywhere."

6.5 ● Medium — CSRF protection is a homegrown, weak implementation

`SecurityMiddleware` issues a CSRF token as `csrf-${Date.now()}-sec-token` if the client doesn't send one, and its validation on mutating requests only checks that **some** `x-csrf-token` header (or `Authorization` header) is present — it never actually validates the token against a server-side expected value or ties it to a session. This is CSRF-shaped code, not CSRF protection. Given the API is JWT-bearer-token-authenticated (not cookie-session-based), classic CSRF risk is already lower, but this middleware currently provides closer to zero real protection and shouldn't be relied on as a compliance control.

6.6 ● Medium — no rate limiting anywhere

No `@nestjs/throttler` or equivalent is present in `apps/api/package.json` or wired into `AppModule / main.ts`. `/api/auth/login` and every other endpoint are unthrottled — add rate limiting (especially on login) before production, particularly since JWT secrets and seeded passwords are the other two findings above.

6.7 Good things already in place (don't regress these)

- `Global ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true })` — solid input hygiene.
- `SecurityMiddleware` does set real security headers (HSTS, CSP `default-src 'self'`, `X-Content-Type-Options`, `X-Frame-Options: DENY`).
- `AllExceptionsFilter` global exception handling.
- `JwtAuthGuard` + `RbacGuard` as global `APP_GUARD`s, with a documented, tested `PERMISSION_GRANTS` matrix in `seed.ts` (13 system roles).
- `AuditInterceptor` logs all mutating actions — good for compliance, just needs a retention plan (§2.2).
- `scripts/mask-staging-data.ts` exists for masking PII before loading prod dumps into staging — make sure this is actually wired into whatever process refreshes staging data, since nothing in the code calls it automatically.

7. Observability & operations

7.1 Health checks

`GET /api/health` (`HealthController`, `@Public()`) returns `{ status, timestamp, uptime, version }`. It is a **liveness-only** check — no DB or Redis connectivity is verified (the module's own comment says Terminus-based dependency checks are planned but not yet implemented, despite `@nestjs/terminus` already being a dependency). Use this for container liveness probes; don't rely on it to catch a dead database connection — `PrismaService.onModuleInit()` currently only **logs a warning** and continues starting up if the initial DB connection fails, it doesn't crash the process.

7.2 Logging

`pino` and `pino-pretty` are dependencies but **not wired into Nest anywhere** (`app.useLogger`, `nestjs-pino`, etc. — none found). The app currently uses Nest's default console `Logger`. Either wire up structured logging for production (recommended for a system handling PHI, where log aggregation/audit matters) or drop the unused `pino` dependencies.

7.3 Error tracking / APM

No Sentry or equivalent APM/error-tracking SDK is installed (`SENTRY_DSN` is commented out and unreferenced in code). For a production healthcare system, decide on this before go-live rather than after the first unreported outage.

7.4 CI/CD — current state

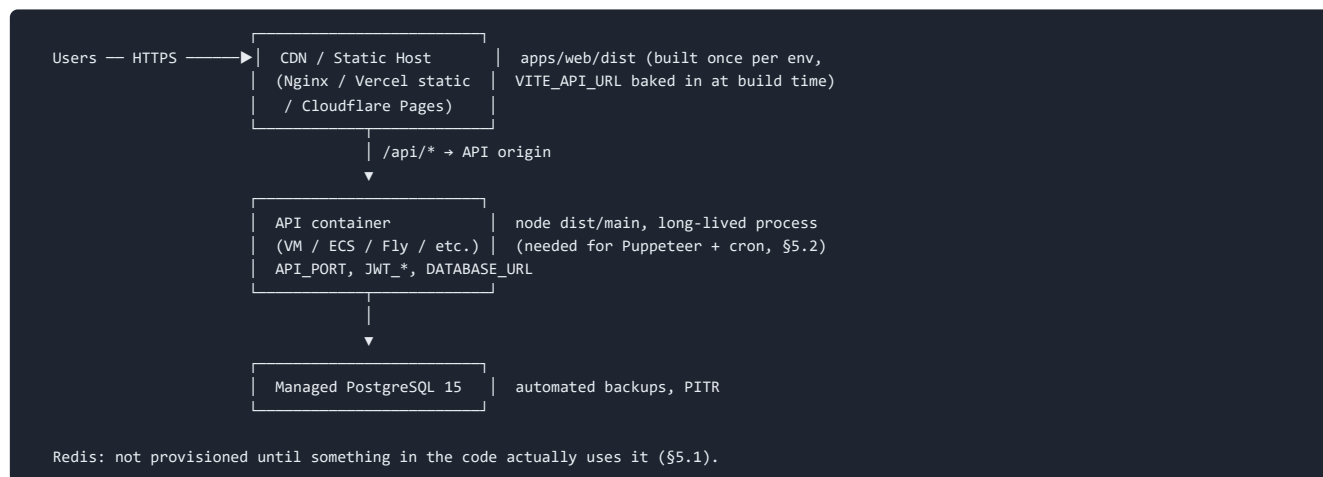
`.github/workflows/ci.yml` runs **lint, typecheck, and test** (with Postgres + Redis service containers) on push/PR to `main`. **There is no deployment/CD step anywhere in the repo** — no build-and-push-image job, no deploy trigger. Standing up actual CD (build Docker images → push to a registry → deploy) is a prerequisite for any repeatable production release process; right now, "deploy" is a manual, undocumented process.

8. Database operations

- **Migrations:** 11 migrations exist under `apps/api/prisma/migrations`. Run `prisma migrate deploy` (never `migrate dev`) against production. This is not part of the app's `build` script — it must be an explicit, ordered step in your release process, run once against the target DB before the new API version starts serving traffic.
- **Seeding:** `prisma db seed` runs `apps/api/prisma/seed.ts`, which seeds **13 RBAC roles + their permission matrix (needed) and demo user accounts with hardcoded passwords (do not run in prod — see §6.2)**. Consider splitting these into two scripts: a prod-safe roles/permissions seed, and a dev-only demo-accounts seed.
- **Backups:** nothing in the repo defines a backup strategy — this is entirely an infra-layer decision (managed Postgres provider snapshots, `pg_dump` cron, etc.). Given this stores patient/employee health records, treat backup + point-in-time recovery as non-negotiable before go-live, not a nice-to-have.
- **Growth driver:** the `AuditInterceptor` writes a row per mutating request with no visible pruning/archival job — budget storage growth and an eventual retention policy accordingly (§2.2).

9. Suggested deployment topology (based on the above)

Given the current code's constraints (persistent Chromium instance, in-process cron, no production web Dockerfile, no CD pipeline):



10. Go-live checklist

- ☐ `JWT_ACCESS_SECRET` and `JWT_REFRESH_SECRET` set to strong random values in the prod environment (never the hardcoded fallbacks)
- ☐ `DATABASE_URL` points at the managed production Postgres instance, with TLS enforced
- ☐ `prisma migrate deploy` run against prod DB as an explicit release step
- ☐ Prod DB seeded with **roles/permissions only** — demo-account seeding (`seed.ts`'s hardcoded passwords) kept out of prod, or immediately rotated if it was run
- ☐ CORS locked to the actual frontend origin(s), not the current open default
- ☐ Rate limiting added to `/api/auth/login` at minimum
- ☐ Production API Dockerfile built (multi-stage, `pnpm build` + `node dist/main`, Puppeteer system libs installed per §5.3) — the current `Dockerfile` is dev-only
- ☐ Production Web Dockerfile/static build pipeline added — the current `Dockerfile` is dev-only (Vite dev server)

- ☐ `VITE_API_URL` set correctly at build time for the target environment before building the web bundle
- ☐ API deployed as a persistent process (container/VM), not Vercel serverless, unless Puppeteer + cron are re-architected first (\$5.2)
- ☐ Backup/PITR strategy configured for PostgreSQL
- ☐ Decision made on Redis: provision it only if/when a feature actually uses it
- ☐ Structured logging and/or an APM/error-tracking tool wired in (currently neither is active)
- ☐ A real CD pipeline added (the existing GitHub Actions workflow is CI-only — lint/typecheck/test, no deploy step)
- ☐ `scripts/mask-staging-data.ts` actually invoked by whatever process refreshes staging from a prod dump